

课程笔记

[CS25 V1] Introduction to Transformers — Instructors

基于公开课程资料整理

2021

Other necessary ingredients for the recipe...

Positional representations/embedding:

- Impart the notion of "ordering", since self-attention is an unordered function of its inputs.

Nonlinearities:

- Implemented as a simple feed-forward network. As with any other deep learning model, this allows for more complex mappings between input and output

Masking:

- In order to parallelize operations while not looking at the future.
- Keeps information about the future from "leaking" to the past
- Used in the decoder (discussed in next slide)

Slide adapted from: [CS224N Winter 2021](#)

Stanford

视频作者/频道: Stanford Online

发布日期: 2021

视频时长: 约 30 分钟

视频链接: https://www.youtube.com/playlist?list=PLoROMvodv4rNiJRchCzutFw5ItR_Z27CM

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1	课程简介	2
2	注意力机制的演进	2
2.1	前 Transformer 时代	2
2.2	早期注意力机制	2
2.3	本章小结	3
3	自注意力与 Transformer 架构	3
3.1	自注意力机制 (Self-Attention)	3
3.2	多头注意力 (Multi-Head Attention)	3
3.3	Transformer 的关键组件	3
3.4	编码器-解码器架构	4
3.5	优缺点分析	4
3.6	本章小结	4
4	基于 Transformer 的里程碑模型	4
4.1	GPT 系列	4
4.2	BERT	5
4.3	本章小结	5
5	Transformer 的未来展望	5
5.1	如何把导论转成后续学习框架	6
5.2	本章小结	6
6	总结与延伸	6
6.1	课程开篇总览表	7
6.2	后续学习路线	7
6.3	拓展阅读	7

1 课程简介

CS25: Transformers United 是由 Stanford 的三位讲师 (Adair、Div、Chaitanya) 于 2021 年秋季创建并教授的一门研讨课。课程主题并非科幻电影中的变形机器人，而是关于深度学习中的 Transformer 模型——一类彻底革新了自然语言处理、计算机视觉、强化学习等多个领域的架构。

课程目标

本课程希望学生达成三个目标：

1. 理解 Transformer 的工作原理；
2. 了解 Transformer 如何被应用于 NLP 之外的领域；
3. 从各位嘉宾的演讲中获得研究灵感。

2 注意力机制的演进

2.1 前 Transformer 时代

在 2017 年 Transformer 出现之前，序列建模的主流架构是循环神经网络 (RNN) 及其变体——LSTM 和 GRU。这些模型擅长编码一定程度的“记忆”，但在处理长序列时存在严重缺陷：

- **长距离依赖问题**：对于句子 “I grew up in France ... so I speak fluent _____”，模型需要从很远处的上下文推断答案为 “French”，但 LSTM 往往做不到。
- **代词消解困难**：当遇到代词 “it” 时，模型难以正确关联到先前出现的名词（如 “animal”）。

RNN 的根本局限

RNN 系列模型按顺序处理序列，信息必须通过隐藏状态逐步传递。序列越长，早期信息的衰减越严重，这是梯度消失问题的直接体现。

2.2 早期注意力机制

在自注意力机制之前，已有几种注意力变体被提出：

- **软注意力 (Soft Attention)**：为图像的每个像素学习一个 $[0, 1]$ 之间的注意力权重。计算量大，但可微分。
- **硬注意力 (Hard Attention)**：直接对感兴趣区域赋值 1，其余为 0。计算更高效，但不可微分，训练更困难。
- **全局注意力 (Global Attention)**：对序列中每个隐藏层输出学习注意力权重 α ，与当前输出做逐元素加权。
- **局部注意力 (Local Attention)**：仅在一个小窗口内计算注意力，而非整个序列。

2.3 本章小结

从简单的软/硬注意力到全局/局部注意力，研究者逐步探索如何让模型”聚焦”于输入中最相关的部分。但这些机制仍然依赖于 RNN 的顺序处理框架，无法从根本上解决长距离依赖和并行化问题。

3 自注意力与 Transformer 架构

3.1 自注意力机制 (Self-Attention)

核心思想：Query-Key-Value

自注意力可以被理解为一个**检索问题**：给定查询 (Query) Q ，找到与之最相似的键 (Key) K ，返回对应的值 (Value) V 。

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中：

- Q ：查询矩阵，表示”我在找什么”
- K ：键矩阵，表示”我有什么可以被查找”
- V ：值矩阵，表示”找到后返回什么”
- d_k ：键向量的维度，用于缩放以防止点积过大

Q 、 K 、 V 三个向量均由同一输入 X 通过不同的线性变换得到，这使模型能够在序列的不同位置之间捕获复杂的交互关系。

3.2 多头注意力 (Multi-Head Attention)

多头注意力是指在每一层中**并行执行多次**自注意力计算，每个”头”可以关注不同的语义信息：

- 一个头可能学习词性 (Part of Speech)；
- 另一个头可能学习句法结构；
- 又一个头可能关注语义角色。

这种设计使模型能够同时学习多个表示子空间。

3.3 Transformer 的关键组件

除自注意力外，Transformer 还依赖以下三个关键组件：

1. **位置编码 (Positional Encoding)**：由于自注意力对所有位置一视同仁，需要额外注入位置信息。RNN 天然按顺序处理，而 Transformer 需要显式的位置表示。
2. **前馈网络 (Feed-Forward Network)**：自注意力本质上是线性运算（矩阵乘法），需要非线性层（MLP）来学习更复杂的输入-输出映射。
3. **掩码机制 (Masking)**：在解码器中，防止当前位置”偷看”未来位置的信息，避免数据泄露。

3.4 编码器-解码器架构

Vaswani 等人 2017 年的论文提出了编码器-解码器架构：

编码器 (Encoder) 由以下子组件构成：

- 多头自注意力层
- 前馈网络层
- 层归一化 (Layer Norm)
- 残差连接 (Residual Connection)

解码器 (Decoder) 与编码器类似，但多了一层**交叉注意力**——它不仅关注解码器前序层的输出，还关注编码器的输出。此外，解码器使用掩码机制确保生成过程的自回归特性。

3.5 优缺点分析

Transformer 的核心优势

- **常数路径长度**：序列中任意两个位置之间的路径长度为 1（每个 token 都直接关注其他所有 token），彻底解决了长距离依赖问题。
- **高度并行化**：不同于 RNN 的顺序处理，Transformer 的计算可以充分利用 GPU 并行能力，训练速度大幅提升。

$O(n^2)$ 的代价

自注意力的时间复杂度为 $O(n^2)$ (n 为序列长度)，因为每个 token 需要与其他所有 token 计算注意力分数。这在长序列场景下成为瓶颈。已有多项工作尝试将其降低到线性或准线性复杂度，如 BigBird、Linformer、Reformer 等。

3.6 本章小结

Transformer 通过自注意力机制实现了”全局视野”，让序列中任意位置都能直接交互，同时借助并行化显著加速训练。其代价是 $O(n^2)$ 的计算复杂度，但这一问题正在被后续研究逐步缓解。

4 基于 Transformer 的里程碑模型

4.1 GPT 系列

GPT (Generative Pre-trained Transformer) 由 OpenAI 提出，仅使用 Transformer 的**解码器**部分，通过自回归语言建模进行预训练——即给定前 t 个 token，预测第 $t + 1$ 个 token。

上下文学习 (In-Context Learning)

GPT-3 展示了一种令人惊讶的能力: 无需梯度更新, 仅通过在输入中提供少量示例 (few-shot), 模型就能“学会”执行新任务。例如:

- 给出几个加法示例后, 模型能完成新的加法计算;
- 展示拼写纠错的样例后, 模型能纠正新的拼写错误;
- 提供翻译对照后, 模型能进行翻译。

GPT-3 的应用包括代码生成 (如 VS Code Copilot)、文本生成、对话系统等。

4.2 BERT

BERT (Bidirectional Encoder Representations from Transformers) 由 Google 提出, 仅使用 Transformer 的**编码器**部分。

BERT 的预训练策略

由于编码器能看到整个序列, 不能使用标准的自回归语言建模 (会导致数据泄露)。BERT 采用了两种预训练任务:

1. **掩码语言建模 (Masked LM)**: 随机遮蔽部分 token, 让模型根据上下文预测被遮蔽的词。
2. **下一句预测 (Next Sentence Prediction)**: 给定两段文本, 判断第二段是否是第一段的后续。

4.3 本章小结

GPT 和 BERT 分别代表了 Transformer 的两种使用范式: 仅解码器 (生成式) 和仅编码器 (判别式)。此后, 领域迅速涌现了 ELECTRA、DeBERTa 等改进模型, Transformer 也被拓展到视觉、语音、蛋白质折叠等多个模态。

5 Transformer 的未来展望

课程讲师们指出了几个值得关注的研究方向:

- **外部记忆**: 类比人脑的海马体, 为 Transformer 引入外部记忆单元 (如 Neural Turing Machine)。
- **线性化注意力**: 将 $O(n^2)$ 的自注意力降至线性复杂度。
- **与人类价值对齐**: 确保语言模型的输出符合人类意图和价值观。
- **跨领域应用**: 视频理解、金融建模、蛋白质结构预测 (AlphaFold)、离线强化学习、文本到图像生成等。

5.1 如何把导论转成后续学习框架

这节导论课的价值不只在回顾 “Attention Is All You Need”，更在于给整门 CS25 建立一张问题地图。后续所有讲次虽然会分别讨论 scaling、alignment、multimodality、biology、robotics 等主题，但都可以回到三个底层问题：

- 1. **表示问题**：模型怎样把序列、图像、动作和结构化对象表示成可计算的 token 或 latent？
- 2. **计算问题**：在上下文越来越长、参数越来越多时，如何控制训练与推理成本？
- 3. **行为问题**：模型学到的不只是统计规律，还会呈现工具使用、规划、偏差和对齐等复杂行为。

CS25 后续主题的阅读坐标

主题簇	需要持续追问的问题	对应能力成长
架构与机制	注意力、位置编码、记忆机制如何改变信息流动？	读懂模型结构与复杂度取舍
预训练与 scaling	数据、算力、参数如何共同塑造能力涌现？	形成系统视角，而不是只看单篇论文
应用与 agent	模型如何从“会补全”走向“会调用工具和执行任务”？	连接研究原理与真实产品
对齐与安全	训练目标与部署目标为何经常不一致？	识别能力增强背后的行为风险

表 1: 把导论课中的核心问题映射到后续课程模块

听这门课的正确姿势

如果只把每一讲当成独立主题，容易得到零散知识点；更有效的做法是始终追问：这一讲是在回答 Transformer 系统中的哪一个瓶颈？它改变的是表示、训练、推理，还是对齐？这样看完整门课后，才能形成真正可迁移的模型直觉。

5.2 本章小结

Transformer 的未来并不只是在 NLP 中继续堆大模型，而是在效率、记忆、对齐和跨模态应用上持续扩展。课程把这节开篇课的意义放得很明确：理解 Transformer，不只是理解一种架构，而是理解未来十年大模型技术栈的共同语言。

6 总结与延伸

本讲作为 CS25 系列的开篇，从注意力机制的历史演变讲起，系统介绍了自注意力和 Transformer 架构的核心原理，并以 GPT 和 BERT 为例展示了 Transformer 的两种基本范式。

Transformer 的成功源于两个关键设计：（1）自注意力机制带来的全局信息交互能力，解决了 RNN 的长距离依赖瓶颈；（2）高度可并行的计算结构，使得大规模预训练成为可能。

6.1 课程开篇总览表

主题	本讲给出的核心答案	对后续课程的意义
为什么需要注意力	让模型按需聚焦输入的关键部分	为自注意力和 Transformer 奠定直觉基础
为什么 Transformer 取代 RNN	解决长依赖与并行训练问题	解释大规模预训练为何可行
GPT 与 BERT 有何不同	分别代表生成式和判别式范式	为后续语言模型与编码模型路线做铺垫
未来研究方向是什么	效率、记忆、对齐与跨模态扩展	把 Transformer 放入更长的研究时间线中

表 2: CS25 Lecture 1 的核心问题与答案

6.2 后续学习路线

1. 先彻底掌握 self-attention 的数学形式和多头注意力的分工
2. 再对照 GPT/BERT 理解 decoder-only 与 encoder-only 的差异
3. 最后把 Transformer 放回更大的模型系统里，关注效率、对齐和跨模态扩展

6.3 拓展阅读

- Jay Alammar, *The Illustrated Transformer*: <http://jalammar.github.io/illustrated-transformer/>
- Vaswani et al., *Attention Is All You Need*, NeurIPS 2017
- BigBird, Linformer, Reformer 等高效 Transformer 论文